---------- Slip 6 ----------

Q.1) Display all the files from current directory which are created in particular month

```
#include <stdio.h>
#include <sys/stat.h>
#include <dirent.h>
#include <time.h>
#include <string.h>

int main() {
    DIR *dir;
    struct dirent *entry;
    struct stat fileStat;
    char monthName[20];

    printf("Enter month name (e.g., Jan, Feb, Mar): ");
    scanf("%s", monthName);

    dir = opendir(".");
    if (dir == NULL) {
        perror("opendir");
        return 1;
    }

    printf("\nFiles modified in %s:\n", monthName);

    while ((entry = readdir(dir)) != NULL) {
        if (stat(entry->d_name, &fileStat) == -1) {
            perror("stat");
            continue;
        }

        // Skip directories (optional)
        if (S_ISDIR(fileStat.st_mode))
            continue;
```

```
        // Convert modification time to string
        char timeStr[100];
        strftime(timeStr, sizeof(timeStr), "%b", localtime(&fileStat.st_mtime));

        // Compare month
        if (strcasecmp(timeStr, monthName) == 0) {
            printf("%s\n", entry->d_name);
        }
    }

    closedir(dir);
    return 0;
}
```

Q.2) Write a C program to create n child processes. When all n child processes terminates, Display total cumulative time children spent in user and kernel mode

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>

int main() {
    int n;
    printf("Enter number of child processes: ");
    scanf("%d", &n);

    pid_t pid;
    struct rusage usage;
    long total_user_sec = 0, total_user_usec = 0;
    long total_sys_sec = 0, total_sys_usec = 0;

    for (int i = 0; i < n; i++) {
```

```
    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // -------------------------
        // Child process: do some work
        // -------------------------
        for (long j = 0; j < 100000000; j++); // CPU-intensive task
        exit(0);
    }
}

// -------------------------
// Parent process: wait for all children
// -------------------------
for (int i = 0; i < n; i++) {
    int status;
    pid_t cpid = wait3(&status, 0, &usage);

    total_user_sec += usage.ru_utime.tv_sec;
    total_user_usec += usage.ru_utime.tv_usec;
    total_sys_sec += usage.ru_stime.tv_sec;
    total_sys_usec += usage.ru_stime.tv_usec;
}

// Normalize microseconds
total_user_sec += total_user_usec / 1000000;
total_user_usec = total_user_usec % 1000000;
total_sys_sec += total_sys_usec / 1000000;
total_sys_usec = total_sys_usec % 1000000;
```

```
    printf("\nTotal cumulative CPU time of children:\n");
    printf("User time: %ld.%06ld seconds\n", total_user_sec, total_user_usec);
    printf("System time: %ld.%06ld seconds\n", total_sys_sec, total_sys_usec);

    return 0;
}
```